Deep Metric Learning for Image Retrieval

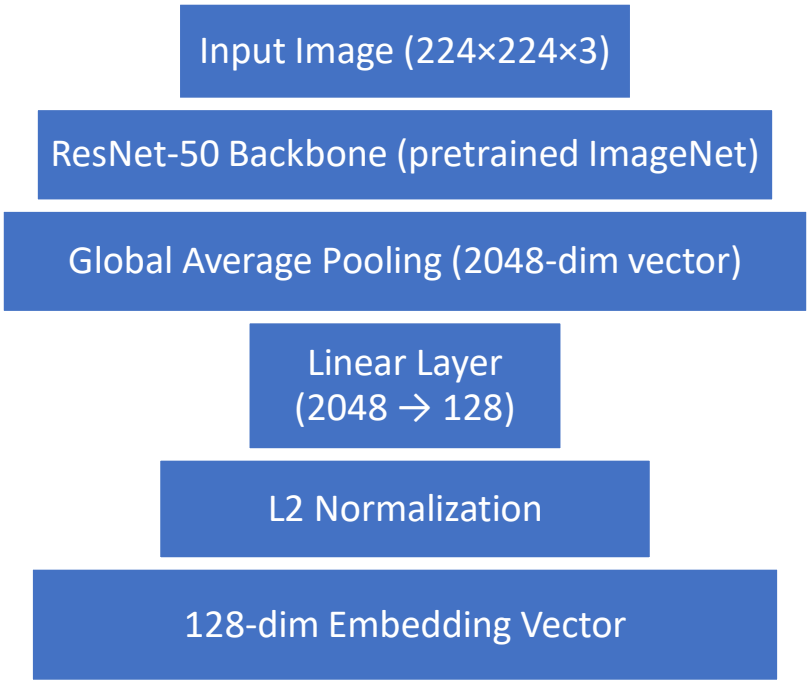
Assignment 3 - Deep Learning Spring 2026

Name: Saad Ali

Roll No: MSDS25066

Model Architecture section:

The embedding network uses ResNet-50 pretrained on ImageNet as the backbone. The final classification layer (fc layer) is replaced with an identity function, exposing the 2048-dimensional global average pooling output. A linear projection layer maps these 2048 features to a 128-dimensional embedding space, followed by L2 normalization which constrains all embeddings to lie on a unit hypersphere.



Design choices to mention:

- Pretrained ResNet-50 chosen for strong visual features
- 128-dim embedding balances expressiveness and efficiency
- L2 normalization ensures distances are bounded and meaningful

- Learning rate 1e-4 chosen for fine-tuning pretrained backbone

Dataset And Sampling Strategy:

Dataset paragraph:

Caltech-101 contains 9,144 images across 102 categories with significant intra-class variation in pose, scale, and illumination. The dataset was split into 6,352 training, 1,326 validation, and 1,466 test images using a stratified 70/15/15 split ensuring all classes appear in every subset.

Contrastive Sampling:

Each training sample returns a labeled pair (image1, image2, label). With probability 0.5 a positive pair is sampled by selecting two images from the same class. With probability 0.5 a negative pair is sampled by selecting images from different classes. This balanced sampling prevents bias toward either pair type.

Triplet Sampling:

Each sample returns a triplet (anchor, positive, negative). The positive is a randomly selected image from the same class as the anchor. The negative is randomly selected from a different class.

Hard Negative Mining:

Instead of random negatives, batch hard mining selects the most informative triplets within each batch. For each anchor, the hardest positive (same class, maximum distance) and hardest negative (different class, minimum distance) are identified using the full pairwise distance matrix. This forces the model to focus on the most challenging examples.

Loss Functions:

Contrastive Loss:

The formula is:

$$L = y \cdot D^2 + (1 - y) \cdot \max(0, margin - D)^2$$

Where D is the Euclidean distance between embeddings, y=1 for same-class pairs and y=0 for different-class pairs. The margin of 1.0 defines the minimum distance enforced between negative pairs. Positive pairs are pulled together by minimizing D^2, while negative pairs are only penalized when closer than the margin.

Triplet Loss:

The formula is:

$$L = max(0, D(a,p) - D(a,n) + margin)$$

The margin of 0.2 enforces that the anchor must be at least 0.2 units closer to the positive than the negative. When this constraint is already satisfied the loss is zero and no gradient flows.

Hard Mining Loss:

The same triplet loss formula is applied but on hard triplets selected per batch. The hardest positive maximizes D(a,p) and the hardest negative minimizes D(a,n) among valid candidates.

Retrieval performance: Results

Results table:

No of Exp	Experiment	Loss Function	Sampling	Recall@1	Recall@5
Exp1	Contrastive	Contrastive	Random Pairs	85.27%	94.07%
Exp2	Triplet Random	Triplet	Random Triplets	59.41%	77.90%
Exp3	Triplet Hard	Triplet	Hard Mining	80.42%	90.79%

Training curves analysis:

Exp1 showed consistent loss reduction from 0.091 to 0.022 over 10 epochs with Recall@1 improving from 97.2% to 98.2% on validation. The model converged smoothly without instability.

Exp2 exhibited triplet collapse beginning at epoch 3, where batch-level loss frequently dropped to zero. This indicates most random triplets became trivially satisfied after early training. Despite near-zero loss, Recall@1 degraded from 96.6% at epoch 1 to 93.4% at epoch 10, demonstrating that zero loss does not mean good embeddings.

Exp3 maintained non-zero loss throughout training (0.076 to 0.030), confirming that hard negatives provide consistent gradient signal. Recall@1 remained stable between 96-98% during training.

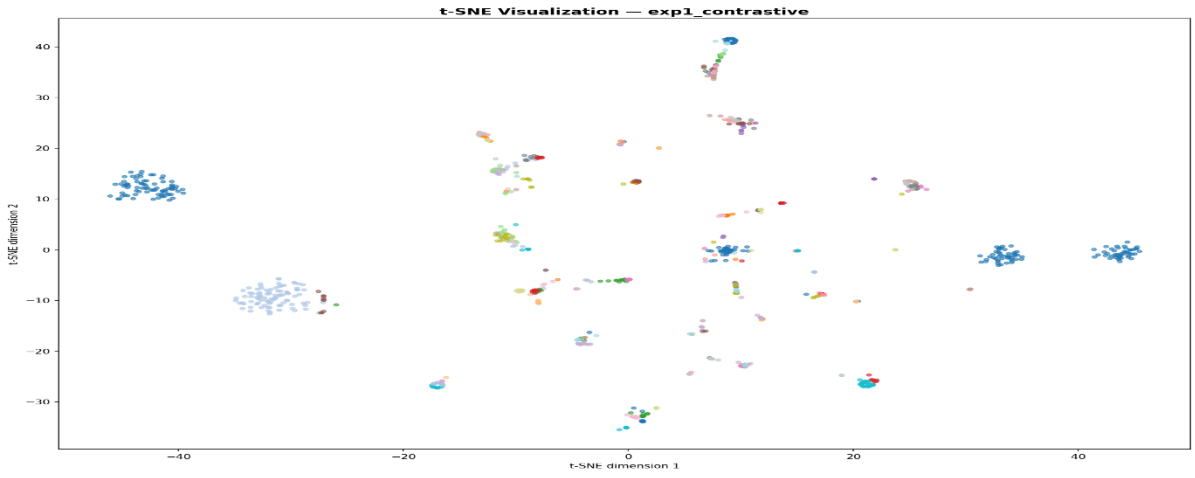
Learning rate discussion:

A learning rate of 1e-4 was selected for fine-tuning. Higher rates (1e-3) risk disrupting pretrained Res-Net features. Lower rates (1e-5) converge too slowly. The chosen rate allowed stable optimization of both backbone and projection head.

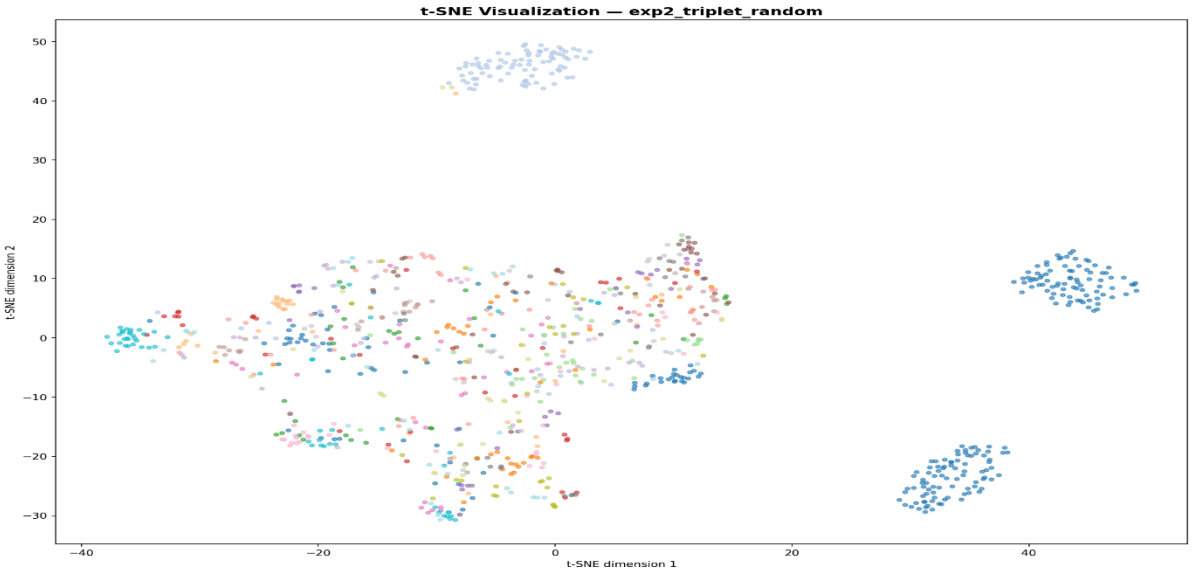
Visualizations:

t-SNE section:

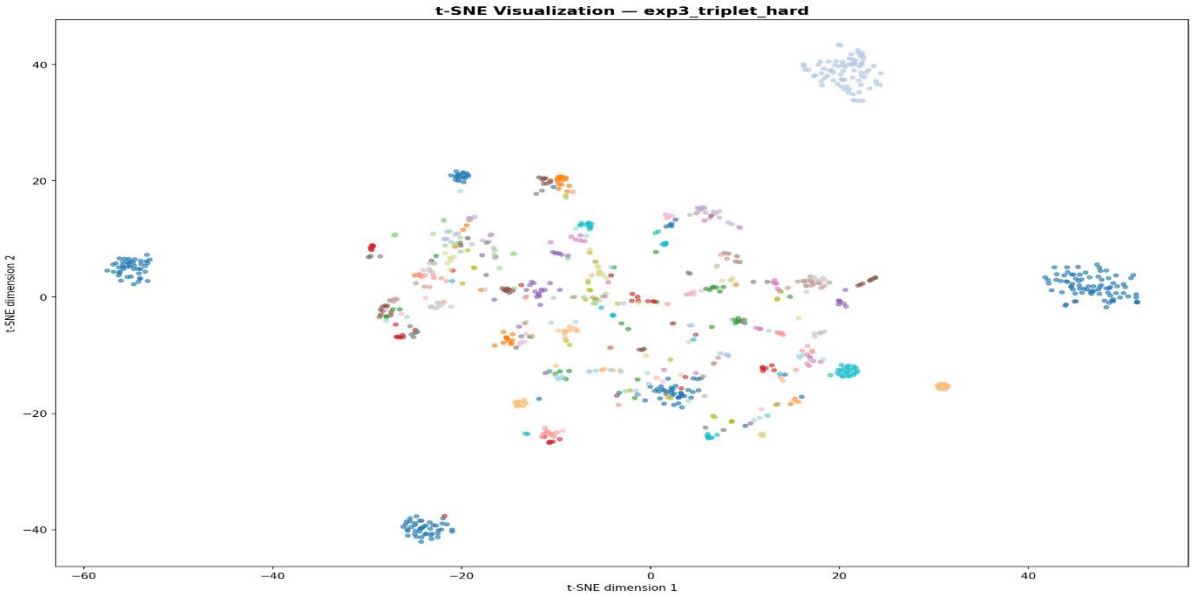
Plot 1 (Contrastive): Classes form distinct, well-separated clusters indicating the contrastive loss effectively organized the embedding space. Similar categories such as Faces and Faces-easy appear close together.



Plot 2 (Triplet Random): Cluster separation is visibly weaker compared to contrastive loss. Several classes overlap, consistent with the lower Recall@1 of 59.41%. The triplet collapse caused the model to stop learning meaningful structure.



Plot 3 (Triplet Hard): Clusters are more defined than random triplets but slightly less clean than contrastive. Hard mining prevented full collapse and maintained reasonable separation.



Retrieval visualization section:

Faces (Exp1): Perfect retrieval — contrastive loss correctly identifies all neighbors as Faces despite variation in background and lighting.

Faces-easy (Exp1): Perfect retrieval — clean frontal face images with minimal background made this an easy case for the embedding.

Motorbikes (Exp1): Perfect retrieval — model correctly groups motorbikes despite variation in color, style, and background.

Faces (Exp2): Perfect retrieval — even random triplets handled the visually distinctive Faces class correctly.

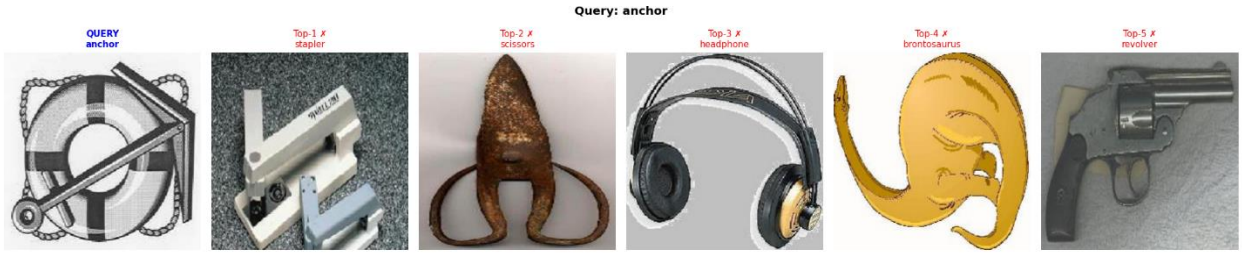
Faces-easy (Exp2): Perfect retrieval — clean centered faces remain easy to retrieve even with the weaker random triplet embeddings.

Motorbikes (Exp2): Perfect retrieval — motorbikes' distinctive shape makes them retrievable even with collapsed random triplet embeddings.

Faces (Exp3): Perfect retrieval — hard negative mining produces equally strong face retrieval as contrastive loss, correctly handling gender and background variation.

Faces-easy (Exp3): Perfect retrieval — interestingly Top-2 through Top-5 cluster around the same red-haired individual, showing hard mining learned fine-grained facial similarity.

Motorbikes (Exp3): Perfect retrieval — hard mining correctly retrieves diverse motorbike styles ranging from cruisers to dirt bikes, showing robust shape-level understanding.



Anchor (Exp3): Complete failure — 0/5 correct. The anchor class contains stylized diagram-style images that share geometric and curved shape features with unrelated objects like scissors, headphones, and brontosaurus, causing total retrieval breakdown

Discussion And Conclusions:

Comparison of methods:

Contrastive loss outperformed both triplet variants on this dataset, achieving 85.27% Recall@1 compared to 80.42% for hard mining and 59.41% for random triplets. The pair-based approach with margin=1.0 provided stable gradients throughout training without collapse.

Random triplet sampling confirmed the known limitation of metric learning — easy triplets dominate training and gradients vanish. Hard negative mining partially addressed this by maintaining non-zero loss, but batch-level mining with batch size 64 cannot find truly global hard negatives, limiting its effectiveness.

Failure cases:

Retrieval failures occur primarily between visually similar categories such as Faces/Faces-easy, cougar-body/cougar-face, and background images with cluttered scenes. These categories share low-level visual features that even deep embeddings struggle to distinguish.

Conclusions:

Deep metric learning with pretrained ResNet-50 achieves strong retrieval performance on Caltech-101. Contrastive loss provides the most stable training. Hard negative mining improves over random triplets but requires larger batch sizes to find truly challenging negatives. Future work could explore online hard mining with larger batches or more sophisticated sampling strategies.

Links of GIT, Google Drive and ChatGPT:

The ChatGPT link is: https://chatgpt.com/c/69b59d6a-5b14-8320-a157-d1b174517ce9

Google Drive link for weights:
https://drive.google.com/drive/folders/1vGg60PnIW5PDYEejfColCRCuxUrKKyLX?usp=drive_link

Google Colab Link: As, my system take too much time in experiment, as you know there are 3 exp with 10 epoch each, So, I use google colab but my files are in .py types. I only use Google colab as machine and export my python code there to run the experiment

https://colab.research.google.com/drive/1qJPgGJNlKj7baenY44ylakOF-hktUVYa?usp=sharing

GIT Repository: https://github.com/Hashimi321/Deep-Metric-Learning-for-Image-Retrieval